

Avances del proyecto de facturación electrónica

Informe funcional y técnico

Fecha de corte 1 de octubre de 2026

Estado general

El proyecto ya cuenta con la estructura visual compartida, la gestión de clientes y el recorrido de facturación desde la selección del comprobante hasta su consulta posterior. Existen vistas para UI-01 a UI-20, además del detalle de eventos de bitácora. El desarrollo sigue en la etapa de frontend con datos definidos en código y operaciones guardadas en memoria.

El avance permite probar navegación, formularios, cálculos, estados de pantalla y exportaciones locales. La emisión fiscal, la firma con certificados, la comunicación con la DGII y la persistencia definitiva todavía requieren integración de backend. La presencia de una pantalla no significa que todas sus acciones estén terminadas.

Indicador comprobado	Estado al corte
Pantallas principales	UI-01 a UI-20 con vistas implementadas
Clientes iniciales	44 registros · 42 activos y 2 inactivos
Facturas iniciales	128 registros · 119 aprobados y 9 rechazados
Bitácora y marco legal	234 eventos de referencia y 8 entradas normativas
Pruebas de frontend ejecutadas	170 aprobadas · Sin acceso a SQL
Comprobación de Django	Sin incidencias detectadas por manage.py check

Qué contiene este documento

Se documentan las funcionalidades de cada pantalla, los componentes reutilizables, el manejo de datos, las validaciones y las comprobaciones realizadas. También se identifican acciones parciales y trabajo pendiente para que el informe sirva como registro de avance y base de planificación.

El inventario corresponde al código local, incluidos cambios aún no registrados en Git. El README describe un estado anterior y todavía presenta UI-12 a UI-19 como pendientes; las rutas y vistas actuales ya contienen esas pantallas.

1 Acceso y gestión de clientes

UI 01 Inicio de sesión

Formulario con validación de credenciales, mensajes de error, visualización de contraseña y retorno a la sección solicitada. El acceso local usa el usuario user y la contraseña 1234, habilitados únicamente cuando DEBUG y FRONTEND_AUTH_ENABLED están activos. La identidad se conserva en una cookie firmada, sin crear usuarios ni sesiones SQL.

Se protegen los destinos de redirección, se rota el token CSRF al acceder y el cierre de sesión usa POST. La identidad local muestra el rol Administrador, pero mantiene is_staff e is_superuser desactivados; no equivale a acceso administrativo de Django.

UI 02 Inicio

Incluye saludo según la hora, identificación del usuario, cuatro tarjetas de resumen y las cinco facturas más recientes. Los indicadores se obtienen del repositorio compartido y reflejan cambios de estado de clientes y nuevas facturas locales. Ofrece accesos a clientes, creación de facturas, historial y detalle.

UI 03 Directorio de clientes

Búsqueda por nombre o identificación con normalización de acentos y separadores; filtros Todos, Activos e Inactivos; contadores y paginación de seis registros. La búsqueda y selección se resuelven en Python. El navegador actualiza resultados, administra cargas y conserva parámetros de navegación.

La tabla muestra identificación, nombre, dirección, teléfono y estado. El menú permite consultar, editar, iniciar una factura y cambiar el estado con confirmación. Se distinguen un directorio sin registros y una búsqueda sin coincidencias.

UI 04 Crear cliente

Registro de RNC o cédula, razón social o nombre, teléfono, correo y dirección. La identificación y el nombre son obligatorios; los contactos son opcionales. Detecta documentos repetidos, incluso en clientes inactivos. El alta se incorpora al repositorio en memoria y vuelve al directorio sin filtrar automáticamente por el documento recién creado.

UI 05 Detalle de cliente

Presenta identificación, contactos, domicilio, fechas de registro y modificación, estado y facturas recientes del receptor. Incluye estados sin facturas, información de contacto faltante y cliente inactivo. Las acciones de inactivar y reactivar conservan las facturas y solicitan confirmación; crear factura depende de que el cliente esté activo.

UI 06 Editar cliente

Carga los datos existentes, indica cambios pendientes y valida el formulario antes de actualizar. Permite mantener el documento propio sin producir un duplicado y rechaza identificaciones de otros clientes. Reutiliza las máscaras numéricas y regresa al detalle después de guardar. Los rótulos de padrón presentes en el diseño no proceden de una consulta fiscal real.

2 Preparación de una nueva factura

UI 07 Tipo de comprobante

Permite escoger E31 o E32 mediante tarjetas y consultar sus diferencias. Crea un borrador con UUID en memoria y conserva el cliente recibido desde su ficha. Valida el tipo seleccionado y evita iniciar el flujo con un cliente inactivo.

UI 08 Selección del cliente receptor

Muestra receptores en tarjetas con búsqueda, seis registros por página, resumen del cliente elegido y controles de avance. La implementación actual restringe E31 a RNC y E32 a cédula; esta regla deberá contrastarse con los requisitos definitivos al integrar el backend. Impide avanzar sin receptor o con uno inactivo o incompatible.

El modal de alta rápida permite crear un cliente sin abandonar el asistente y aplica las validaciones y detección de duplicados del formulario general. La selección se guarda en el mismo borrador para conservar la continuidad entre pasos.

UI 09 Datos generales

Recoge fecha de emisión, tipo de ingreso, modalidad de pago, medio de pago y vencimiento cuando corresponde. Admite contado, crédito y gratuito, así como plazos predefinidos y fecha personalizada. El formulario comprueba que el vencimiento de crédito no anteceda a la emisión y limpia los datos de crédito al cambiar de modalidad.

Las tarjetas de contexto mantienen visibles tipo y receptor. Los enlaces permiten corregir pasos anteriores. La secuencia mostrada durante la preparación es orientativa; aún no existe asignación oficial de e-NCF.

UI 10 Bienes y servicios

Editor de líneas para bienes y servicios con concepto, descripción, cantidad, unidad, precio y descuento. Permite agregar, modificar y quitar conceptos con confirmación. La vista previa se actualiza mediante solicitudes al servidor y diferencia los cambios guardados de los pendientes.

Los importes usan Decimal y redondeo a centavos. El descuento se aplica antes del ITBIS, fijado en 18% en este flujo. Se admiten hasta 100 líneas; los límites numéricos, las opciones válidas y el descuento máximo se comprueban en el servidor. La revisión de líneas evita sobrescrituras y envíos repetidos desde formularios desactualizados.

UI 11 Totales y condiciones

Resume subtotal bruto, descuentos, base imponible, ITBIS y total a pagar. Muestra las condiciones comerciales, fechas, desglose tributario y detalle de conceptos en una ventana. El guardado registra fecha y huella del borrador en memoria.

Si cambian los conceptos o las condiciones después de guardar, exige recalcular y confirmar. Los POST vuelven a validar y calcular; los importes enviados por el navegador no se consideran la fuente de verdad. Las huellas permiten detectar cambios entre pestañas antes de continuar a la revisión final.

3 Revisión y consulta del comprobante

UI 12 Revisión final

Presenta una hoja de factura con emisor, receptor, conceptos, condiciones, totales y monto en letras. Incluye inspección del XML, copia de contenido, impresión y guardado del borrador. Comprueba que existan cliente activo, datos generales, líneas y totales confirmados, y que la huella siga correspondiendo al borrador.

El checklist y los datos de certificado de esta vista contienen información de referencia. Sus mensajes de validación fiscal o firma no acreditan una verificación externa ni una firma electrónica real.

UI 13 Emisión y procesamiento

La acción de emitir por POST crea una Factura en el repositorio local, le asigna un e-NCF calculado por conteo, un identificador de seguimiento y estado aprobado. Conserva líneas y condiciones y conduce a la pantalla de procesamiento. Esta operación no transmite el comprobante a la DGII.

La pantalla representa seis fases: preparar, XML, validación, firma, envío y acuse. Incluye vistas de finalización, error de validación y tiempo de espera agotado. El progreso se controla con JavaScript y temporizadores; todavía no está conectado a una cola de trabajos ni a un servicio de seguimiento real.

UI 14 Resultado de emisión

Muestra aprobación o rechazo, e-NCF, identificador de seguimiento, desglose de importes y respuesta XML de referencia. Permite copiar información y consultar el dictamen. Los motivos de rechazo se construyen localmente y algunos escenarios se seleccionan mediante parámetros de URL.

UI 15 Detalle de factura

Expediente con emisor, receptor, fechas, condiciones, conceptos, totales y monto en letras. Las facturas creadas desde el asistente conservan sus líneas. Incluye impresión, copia del e-NCF y acceso a los documentos. Existen presentaciones de aprobado, rechazado y anulado; mostrar una anulación no implementa su trámite fiscal.

UI 16 Documento XML

Visor con líneas numeradas, tamaño, estado y datos asociados. Genera XML de referencia y permite copiarlo o descargarlo como archivo desde el navegador mediante Blob. Los valores de firma y certificado son ilustrativos y no proceden de una clave privada o validación XSD. La ruta directa de descarga del servidor sigue respondiendo HTTP 501.

UI 17 Representación impresa

Vista HTML con apariencia de papel, detalle de ítems, montos, receptor y emisor. Incluye zoom, ajuste al ancho, guías y cuadrícula. Imprimir y el botón de descarga abren window.print; el usuario puede guardar como PDF desde el navegador. La generación de un PDF descargable en el servidor sigue pendiente y su ruta devuelve HTTP 501.

4 Historial y herramientas de consulta

UI 18 Historial de facturas

Archivo de comprobantes con vista dividida y vista de tabla. La búsqueda admite e-NCF, identificación del receptor, nombre y TrackId. Ofrece filtros por tipo de comprobante, estado y fechas, además de accesos rápidos para E31, E32, aprobados y rechazados.

La paginación admite 8, 10, 20 o 50 registros por página. El panel lateral muestra el expediente seleccionado, receptor, emisor, ítems y accesos relacionados. Contempla historial vacío y búsquedas sin resultados. La exportación CSV incluye el conjunto filtrado, con datos de identificación, fecha, total, estado y seguimiento.

Los filtros se ejecutan en Python sobre los registros locales. Todavía no son consultas SQL ni existen mediciones de rendimiento con grandes volúmenes. Los comprobantes de referencia sin líneas completas usan desgloses contruidos para su presentación.

UI 19 Bitácora y detalle del evento

Consulta de 234 eventos iniciales con fecha y hora, usuario, módulo, descripción, referencia y resultado. Incluye búsqueda de texto, filtros por módulo, usuario, resultado y fechas, etiquetas de filtros activos y diez eventos por página.

La pantalla de detalle expone datos técnicos en JSON, códigos, hash e información de seguimiento, con referencia a factura o cliente cuando existe. Se pueden copiar contenidos y exportar registros filtrados a CSV. Las vistas solo aceptan lectura; un POST al listado devuelve HTTP 405.

Los eventos provienen de fixtures. La existencia de hashes, direcciones IP y tiempos de respuesta en estas filas no constituye un registro de actividad real: todavía falta enlazar la generación de eventos a las operaciones del sistema y almacenarlos de forma persistente.

UI 20 Marco legal

Sección con ocho entradas de leyes, normas, estándares y decretos definidos en `core/legal_data.py`. Incluye clasificación, conteos, resúmenes, artículos, impacto en el sistema y referencias. El servidor admite búsqueda y filtro por tipo; JavaScript ofrece búsqueda inmediata sobre las tarjetas cargadas.

Esta funcionalidad organiza contenido informativo del proyecto. Su implementación no equivale a una validación jurídica, a una comprobación de vigencia normativa ni a una certificación del sistema.

Navegación y estados transversales

Las pantallas comparten navegación privada, migas de pan, mensajes, indicadores de estado e iconos. Se añadieron paginación asíncrona, esqueletos de carga y soporte del historial del navegador. Hay plantillas para errores 400, 403, 404 y 500, y respuestas 501 para descargas aún reservadas.

5 Arquitectura y sistema visual

Organización del código

La aplicación utiliza Django 6.0.8, plantillas HTML, CSS y JavaScript nativo. Los módulos separan acceso, clientes, comprobantes, bitácora y elementos comunes. El recorrido principal funciona con un repositorio Python en memoria y no depende de los modelos SQL para sus datos de negocio.

Capa o módulo	Responsabilidad
facturacion_ecf	Configuración, rutas raíz, middleware y archivos estáticos.
core y templates	Inicio, marco legal, navegación, layouts, iconos y errores.
usuarios y clientes	Acceso local, formularios y gestión del receptor.
comprobantes	Asistente, cálculos, emisión local, documentos e historial.
bitacora	Consulta, detalle y exportación de eventos.
frontend_data	Entidades, datos iniciales y operaciones del repositorio.
static	CSS global, ajustes por pantalla y comportamiento JavaScript.

Datos compartidos y consistencia

Las entidades Cliente, Factura, BorradorFactura, LineaFactura y EventoLog se representan con dataclasses. El repositorio mantiene índices por identificación y facturas por cliente, y utiliza RLock en operaciones de escritura. Los borradores tienen UUID; las líneas y los totales incluyen controles de revisión y huella.

El estado se conserva mientras el proceso siga activo. Reiniciar el servidor o recargar código elimina altas, modificaciones, borradores y nuevas emisiones locales. Los datos tampoco se sincronizan entre procesos. Esta limitación define el alcance de la etapa actual.

Estilos y componentes reutilizables

global.css centraliza tokens, base, layouts, componentes y utilidades mediante capas CSS. Define azul marino #082B53, fondo #F6F8FD, superficies blancas, colores semánticos, espaciado, bordes, sombras, foco y puntos de adaptación a móvil. La tipografía principal es Plus Jakarta Sans, con alternativas del sistema.

Cada pantalla agrega sus estilos en static/css/pages y su comportamiento en static/js/pages. Se reutilizan botones, tarjetas, badges, formularios, tablas, diálogos, pasos del asistente y formato de importes. La base carga la fuente desde Google Fonts.

Base de backend ya existente

Existen modelos de Usuario con roles, Emisor, Certificado, Receptor, Comprobante, ItemComprobante, ColaContingencia y LogTransaccion, junto con migraciones iniciales y un generador XML básico. Son una base de trabajo; su presencia no significa que estén conectados al recorrido actual ni que implementen firma, contingencia o integración fiscal.

6 Validaciones y experiencia de uso

Entrada de datos del cliente

- RNC de 9 dígitos y cédula de 11, con rechazo de letras en campos numéricos y formato automático. Ejemplo de cédula: 402-7887785-7.
- Teléfono de 10 dígitos con máscara progresiva 809-555-1044. El usuario introduce números y el formulario agrega los separadores.
- Normalización de texto y límites de longitud, validación de correo, rechazo de caracteres de control y conservación de caracteres legítimos como apóstrofes.
- Identificación única dentro del repositorio, incluso para clientes inactivos; los errores conservan los valores útiles y señalan los campos a corregir.

Integridad de importes y del asistente

La cantidad y el precio deben ser positivos; el descuento no puede superar el importe bruto de la línea. Los cálculos se realizan con Decimal y las vistas de ítems y totales reutilizan las operaciones del servidor. El ITBIS del flujo actual es fijo al 18%; no existe todavía cobertura completa de exenciones, retenciones u otros tratamientos.

El asistente controla requisitos entre pasos. Ítems y totales detectan formularios obsoletos mediante versiones y huellas. La revisión comprueba que los totales sigan vigentes; el endurecimiento de la acción final de emisión se incluye entre los pendientes de este informe.

Protecciones ya implementadas

Las rutas privadas requieren autenticación. Los formularios de modificación emplean POST y CSRF; hay controles de destino para evitar redirecciones externas y escape HTML en las plantillas habituales. La autenticación local utiliza comparación constante y una cookie firmada, y se desactiva fuera del modo de desarrollo.

Estas medidas cubren partes del flujo y cuentan con pruebas, pero no permiten afirmar que toda la aplicación esté auditada. La protección de SQL deberá basarse en ORM o consultas parametrizadas, validación en servidor y restricciones de datos cuando se conecte la persistencia. Sanitizar texto por sí solo no garantiza esa protección.

Ajustes de uso ya incorporados

- La rueda del mouse deja de incrementar o reducir el campo numérico enfocado; el manejador global también cubre formularios insertados dinámicamente.
- Nuevo cliente incorpora el icono de agregar; al guardar un cliente se vuelve a un directorio sin búsqueda impuesta.
- Confirmaciones para cambios de estado y eliminación de líneas, indicadores de guardado y avisos de cambios pendientes.
- Foco visible, acceso al contenido principal, etiquetas de formularios, atributos ARIA en controles y soporte de Escape y foco en diálogos específicos.
- Navegación móvil, rejillas adaptables y respeto de la preferencia de movimiento reducido en los estilos globales.

7 Verificación del avance

Resultados obtenidos el 1 de octubre de 2026

manage.py check terminó sin incidencias. La selección de módulos SimpleTestCase ejecutó 170 pruebas y finalizó con resultado OK. Estas clases impiden consultas a la base de datos, de modo que verifican el recorrido local sin crear una base de pruebas ni ejecutar migraciones.

Área	Pruebas aprobadas
Repositorio y datos iniciales	7
Inicio y navegación del panel	10
Autenticación local	7
Directorio y gestión de clientes	38
Tipo, receptor y datos generales	39
Ítems y totales	19
Revisión y emisión con procesamiento	23
Resultado, detalle y documentos	20
Historial de facturas	7
Total ejecutado	170

Qué cubren las pruebas

Incluyen acceso privado, errores de identificador, validación de formularios, duplicados, cambios de estado, CSRF, redirecciones, escape de contenido, cálculos decimales, límites de líneas, conflictos entre pestañas, guardado de borradores, emisión local y renderizado de las vistas posteriores.

La cobertura se refiere a esos casos comprobados. No se calculó un porcentaje de cobertura y el resultado no representa una certificación de seguridad, rendimiento, accesibilidad o cumplimiento fiscal.

Comprobaciones adicionales sin SQL

Un cliente de pruebas con bloqueo explícito de consultas confirmó HTTP 200 para listado, búsqueda, filtro, detalle y CSV de bitácora, y para marco legal con filtros y sin resultados. El POST al listado de bitácora devolvió 405. Las rutas directas de descarga XML y PDF devolvieron 501, tal como establece el enrutamiento actual.

Pruebas que quedan fuera de esta ejecución

Hay 41 casos adicionales basados en TestCase en usuarios/tests.py, core/tests.py y bitacora/tests.py. No se ejecutaron en esta revisión porque requieren preparar una base de pruebas. También queda pendiente una pasada integral de navegador sobre las nuevas pantallas, impresión, portapapeles, descargas y convivencia de los manejadores de paginación.

8 Acciones parciales y puntos por resolver

Integración fiscal y persistencia

La prioridad de la siguiente etapa es sustituir el almacenamiento local por persistencia y servicios reales. Faltan el padrón, certificados, firma XML, validación XSD, secuencias autorizadas, transmisión, seguimiento de respuestas y contingencia. También faltan permisos por usuario o emisor y auditoría generada por operaciones reales.

Emisión final y coherencia de datos

emitir_factura compara la huella recibida solo si se suministra, pero no vuelve a calcular la huella actual del borrador. Tampoco consume el borrador ni registra una clave de idempotencia; repetir el POST puede generar otra factura. Antes de integrar emisión real deben reforzarse la validación final, la unicidad y el control de concurrencia.

La operación local usa la hora del servidor como fecha de emisión y asigna aprobado directamente. Resultado, detalle, XML y PDF contienen cálculos duplicados y valores de referencia, incluido un caso especial para RD\$ 336,890.00. Conviene consolidar esas vistas sobre una única instantánea de importes y datos del comprobante.

Documentos y exportaciones

La descarga XML desde el visor funciona en el navegador, pero su endpoint directo permanece reservado. El PDF se obtiene por impresión del navegador. Hay que conectar las descargas definitivas, persistir los archivos y unificar su contenido con la factura guardada.

El XML del visor incorpora textos mediante interpolación sin un serializador XML que escape todos sus valores. Debe comprobarse con nombres y conceptos que contengan símbolos como & o <. Los CSV de historial y bitácora escriben texto sin neutralizar posibles fórmulas de hoja de cálculo; esa protección debe incorporarse antes de exportar datos externos.

Contenido dinámico y lenguaje de la interfaz

La búsqueda de marco legal inserta el término escrito en innerHTML al construir el estado sin resultados. Debe reemplazarse por inserción como texto. Este hallazgo procede de la lectura del código y no de una explotación ejecutada.

Persisten etiquetas de firma válida, certificado autorizado, padrón activo o timbrado que usan valores locales. También existen parámetros de URL que cambian el estado mostrado. Se debe alinear estos mensajes con el estado real del servicio y evitar presentar validaciones no realizadas. Algunas descripciones de fixtures aún contienen la palabra Mock.

Rendimiento y documentación

La búsqueda del directorio y los filtros del historial y bitácora se procesan en el servidor, pero recorren datos en memoria. La futura implementación debe utilizar índices, filtros SQL, paginación y restricciones de unicidad, y verificar planes de ejecución. No se han aplicado optimizaciones SQL en esta etapa.

El README y parte del mapa de pantallas están desactualizados respecto al código. Deben sincronizarse, especialmente UI-12 a UI-20, el rol del usuario local y el alcance de las descargas. Ninguno de estos puntos se modificó durante la elaboración del informe.

9 Mapa de rutas implementadas

Las rutas usan nombres de Django por módulo. Los identificadores de cliente y factura son enteros; los borradores usan UUID. El acceso al contenido de negocio requiere una sesión.

Pantalla o acción	Ruta
UI-01 Acceso	/acceso/iniciar-sesion/
UI-02 Inicio	/inicio/
UI-03 Directorio	/clientes/
UI-04 Alta de cliente	/clientes/nuevo/
UI-05 Detalle de cliente	/clientes/<pk>/
UI-06 Editar cliente	/clientes/<pk>/editar/
Estado del cliente por POST	/clientes/<pk>/estado/
UI-07 Tipo	/facturas/nueva/tipo/
UI-08 Receptor	/facturas/borradores/<uuid>/cliente/
UI-09 Datos generales	/facturas/borradores/<uuid>/datos/
UI-10 Ítems	/facturas/borradores/<uuid>/items/
UI-11 Totales	/facturas/borradores/<uuid>/totales/
UI-12 Revisión	/facturas/borradores/<uuid>/revision/
Emitir por POST	/facturas/borradores/<uuid>/emitir/
UI-13 Procesamiento	/facturas/<pk>/procesamiento/
UI-14 Resultado	/facturas/<pk>/resultado/
UI-15 Detalle	/facturas/<pk>/
UI-16 XML	/facturas/<pk>/xml/
UI-17 Representación impresa	/facturas/<pk>/pdf/
UI-18 Historial	/facturas/
UI-19 Bitácora y evento	/bitacora/ y /bitacora/<pk>/
UI-20 Marco legal	/marco-legal/
Descargas reservadas con 501	/facturas/<pk>/xml/descargar/ /facturas/<pk>/pdf/descargar/

La raíz redirige al inicio; /facturas/nueva/ redirige a la selección de tipo; /logs/ funciona como alias hacia la bitácora. /admin/ corresponde al administrador de Django y no forma parte del acceso local habilitado para frontend.

10 Referencias técnicas y continuidad

Archivos que sustentan el inventario

- Rutas y composición: facturacion_ecf/urls.py, core/urls.py, clientes/urls.py, comprobantes/urls.py, bitacora/urls.py y usuarios/urls.py.
- Datos y operaciones locales: frontend_data/entities.py, fixtures.py y repository.py.
- Clientes y acceso: clientes/forms.py, clientes/views.py, usuarios/forms.py, usuarios/views.py y usuarios/frontend_auth.py.
- Asistente y cálculos: comprobantes/views.py, forms.py, items.py, totales.py y las vistas views_items.py, views_totales.py y views_revision.py.
- Etapas posteriores: comprobantes/revision.py y las vistas views_procesamiento.py, views_resultado.py, views_detalle.py, views_xml.py, views_pdf.py y views_historial.py.
- Consulta transversal: bitacora/views.py, core/views.py y core/legal_data.py.
- Presentación: templates/base.html, layouts, includes, plantillas por módulo, static/css/global.css, static/css/pages, static/js/app.js y static/js/async-pagination.js.
- Evidencia automatizada: módulos tests_* de comprobantes y clientes, frontend_data/tests.py, core/tests_dashboard.py y usuarios/tests_frontend_auth.py.

Cómo ejecutar la aplicación local

Con el entorno ya preparado, ejecutar en PowerShell desde la raíz del proyecto:

```
.\venv\Scripts\python.exe manage.py runserver
```

Abrir <http://127.0.0.1:8000/acceso/iniciar-sesion/> e ingresar user / 1234. Los cambios de datos de esta etapa se pierden al reiniciar el proceso. No se requieren migraciones para el recorrido local descrito.

Orden propuesto para la siguiente etapa

- Cerrar los ajustes funcionales del frontend: consistencia de importes, descargas, enlaces, estados y textos; verificar el recorrido completo en escritorio y móvil.
- Atender los puntos de integridad y contenido dinámico identificados en este informe y ampliar las pruebas de las acciones finales.
- Diseñar la persistencia, los permisos, las relaciones y los índices tomando como contrato las búsquedas, filtros y formularios existentes.
- Conectar autenticación, repositorios, generación documental, servicios fiscales y auditoría; sustituir progresivamente los datos y respuestas locales.
- Preparar la configuración de despliegue y ejecutar pruebas de integración, seguridad y rendimiento antes de usar la aplicación con datos operativos.

El resultado actual es una interfaz funcional extensa con un recorrido local de facturación y una base reutilizable para la integración. Los siguientes hitos deben completar la persistencia y los servicios manteniendo el comportamiento y la línea visual ya construidos.